Stata for Survey Microdata

A Practical Introduction

Alexia Sánchez-Vázquez

Outline

1. Getting started

2. Cleaning and building variables

3. Confidential microdata

Who these slides are for

- Economists who know what a weighted mean, a median or a standard error is, but have **never used Stata**.
- Who need to work with **household survey microdata** such as the Spanish Survey of Household Finances (EFF) or the euro area Household Finance and Consumption Survey (HFCS).
- Every command is explained piece by piece, and the red boxes show the mistakes that almost everyone makes at the start.

Key idea

Read with Stata open. Type every example yourself: most of what you learn comes from the error messages.

Getting started

What is Stata?

- A program for statistical analysis: you give it a dataset and **commands**; it returns tables, statistics and estimates.
- A dataset is a table:
  - each **row** is an *observation* (a household, a person);
  - each **column** is a *variable* (income, age).
- Stata holds **one dataset in memory** at a time. Every command acts on it.
- Nothing you do in memory changes the file on disk until you explicitly save it.

The interface

Window	What it does
Command	Type a command and press Enter.
Results	Output and error messages (in red).
History	Commands run in this session.
Variables	Variables in memory.
Properties	Type, label, format of a variable.

The **Do-file Editor** is the window you will use most: a text editor where you write commands and save them. Open it with the toolbar or by typing `doedit`.

An error is printed in red, followed by a code such as `r(111)`. Click on the code for a longer explanation.

Always work with do-files

- A **do-file** (`.do`) is a text file with commands.
- Run the whole file, or only the selected lines: **Cmd+Shift+D** (Mac), **Ctrl+D** (Windows).

Key idea

Stata has **no undo button**. If you change data by mistake, the only way back is to reload the data and rerun your commands. The do-file is the record that makes this possible, and that lets anyone reproduce your results.

Good practice

When something goes wrong, rerun the do-file **from the top**. Running lines one by one, in a different order each time, is how datasets end up in states nobody can reproduce.

Comments

Comments are notes for humans; Stata ignores them.

```
* A line starting with an asterisk is a comment
clear all          // text after two slashes is a comment
/* Everything between these
   two markers is a comment */
summarize income ///
    age            // three slashes: the command continues on the next line
```

Common error

//, /// and /* */ only work **inside a do-file**. Typed in the Command window they cause an error: join the command into one line and delete the slashes.

The header of every do-file

```
* ------------------------------------------------------------
* Project : practice
* File    : 01_cleaning.do
* Purpose : clean income variables of the practice survey
* Author  : name, date
* ------------------------------------------------------------
version 19                  // follow the rules of this Stata version
clear all                   // empty the memory
cd "/Users/me/project"      // working folder
```

- `version`: the do-file gives the same results in newer versions.
- `cd`: change directory; `pwd` shows the current one.
- Optional: `log using name, replace text` records the output in a file.

Common error

Paths with spaces need double quotes: `cd "/My Documents/project"`.

Anatomy of a command

Almost every command follows the same pattern:

```
command  varlist  if  in  [weight]  ,  options
```

Part	Meaning and example
`command`	What to do: `summarize`, `tabulate`, `list`
`varlist`	Which variables: `income age`
`if`	Which observations: `if country == "ES"`
`in`	Which rows by position: `in 1/10`
`[weight]`	Survey weights, in square brackets: `[aw = weight]`
`,`	Separates the command from its options
`options`	How to do it: `detail`, `replace`, `noobs`

`help` *command* shows the exact syntax: parts in square brackets are optional; anything not listed is **not allowed**.

Reading a command

```
summarize inc_hh if country == "ES" [aw = weight], detail
```

- `summarize`: compute summary statistics,
- `inc_hh`: of household income,
- `if country == "ES"`: only for Spanish households,
- `[aw = weight]`: each household weighted by `weight`,
- `, detail`: and also report percentiles, not only the mean.

Good practice

Before running a long command, read it aloud in this order. If one piece has no meaning, the command is probably wrong.

When a command does not fit its syntax

Common error

- `if not allowed r(101)`: the command takes no `if` (for example `describe`).
- `too many variables specified r(103)`: `tabulate` takes at most two variables.
- `option ... not allowed r(198)`: that option does not exist for that command.

Key idea

To **see values** of some variables for some rows, use `list`: `list hh_id income if hh_id == 3`. `display` is not for variables: it shows one number, text or calculation, such as `display 2 + 2`.

Loading and saving data

```
sysuse auto, clear            // built-in example dataset
use "mydata.dta", clear       // your own file; clear = drop what is in memory
save "mydata_clean.dta", replace   // replace = overwrite if it exists
```

Common error

- `no; dataset in memory has changed since last saved r(4)`: add `clear` to `use`.
- `file ... already exists r(602)`: add `replace` to `save`.

Be careful

Never use `save ..., replace` on an original data file.

Looking at the data

Command	Question it answers
`describe`	How many observations and variables? Text or numbers?
`browse`	What do the data look like? (read-only spreadsheet)
`list x y in 1/5`	What are the values in the first five rows?
`codebook x`	What type is x, how many values are missing?
`summarize x`	Number of observations, mean, sd, min, max.
`tabulate x`	How many observations in each category?
`count if ...`	How many observations meet a condition?

Common error

= assigns, == compares. `summarize x if country = "ES"` gives `invalid syntax r(198)`.

Survey microdata: three things that matter from day one

1. **Weights.** Some households are deliberately oversampled (the wealthy, in wealth surveys). Each household has a weight: how many households of the population it represents. Without weights you describe the sample, not the population.
2. **Missing values**, written as a dot (`.`). A missing value can mean "does not have it", "does not know" or "question not asked". Each requires a different treatment.
3. **The unit of observation.** What is a row: a person, a household? Surveys come in several files with different units, linked by an identifier.

Be careful

For Stata, a missing value is **larger than any number**. `if income > 1000` also selects missing incomes. Write `if income > 1000 & !missing(income)`. Stata gives no error: the result is simply wrong.

The EFF and the HFCS

- Surveys of household wealth, debt, income and consumption, run by the Banco de España (EFF) and the Eurosystem (HFCS).
- Available to researchers on request, under confidentiality rules.
- Two extra features (Part 3):
  - **Multiple imputation**: unknown values are filled in five times, so each household appears five times, once per *implicate*.
  - **Replicate weights** (HFCS): 1,000 extra weights per household, for standard errors.

Good practice

Never modify the original data files. Read from them, save your work in another folder, and note in the header which file and release you used.

Cleaning and building variables

The practice dataset

```
clear
input hh_id person_id str2 country weight age inc_wage inc_pension owns_home home_value
1 1 "ES" 1500 45 28000     .  1 220000
1 2 "ES" 1500 43    -1     .  1 220000
2 1 "ES" 2100 71     .  14000 1 150000
3 1 "PT"  900 33 21000     .  0     -2
3 2 "PT"  900 30 19000     .  0     -2
4 1 "PT" 1200 58    -1    -1  1     -1
5 1 "ES"  800 67     .  11000 0     -2
5 2 "ES"  800 65     .   9000 0     -2
6 1 "PT" 1700 52 35000     .  1 180000
end
```

- One row per **person**; `weight`, `owns_home`, `home_value` belong to the household and repeat.
- Codes: `-1` = does not know; `-2` = not applicable; `.` = does not receive that income.
- `str2`: `country` is text, two characters long.

First question: is the mean wage right?

`summarize inc_wage` gives a mean of 17,166 over 6 observations. It is wrong for three reasons:

1. The two `-1` are counted as wages of -1 euro.
2. The three dots mean "no wage", but `summarize` drops them: the mean is only among earners.
3. No weights: it describes the sample, not the population.

Key idea

Before computing anything, decide what each code means:

- "does not have it" $\Rightarrow$ **0**;
- "does not know" $\Rightarrow$ **missing** (`.`);
- "not applicable" $\Rightarrow$ depends on the statistic (0 for total wealth).

generate and replace

```
generate inc_k = inc_wage / 1000                                    // new variable
generate high_wage = (inc_wage > 30000) if !missing(inc_wage)       // dummy 0/1
replace  inc_k = round(inc_k)                                       // change an existing one
```

- `generate` creates a **new** variable; `replace` changes an **existing** one.
- `(condition)` equals 1 if true and 0 if false: the quickest dummy.

Common error

- `variable ... already defined r(110)`: `generate` on an existing variable. Use `replace` or `drop` it first.
- `variable ... not found r(111)`: misspelt or non-existent variable. Upper and lower case differ.

Assigning a missing value vs asking about it

```
replace inc_wage = . if inc_wage == -1          // correct: the dot assigns a missing value
replace inc_wage = missing(inc_wage) if inc_wage == -1   // WRONG
```

Common error

`missing(x)` is a **question**: it equals 1 if `x` is missing and 0 otherwise. In the second line, for rows with -1 the answer is "not missing", so the -1 becomes **0**. Stata gives no error.

Key idea

- To **assign** a missing value: `= .`
- To **ask** whether a value is missing: `missing(x)` in an `if`, or inside a calculation.

Order matters

Goal for `inc_wage`: dots (no wage) → 0; -1 (does not know) → missing.

```
replace inc_wage = 0 if missing(inc_wage)     // first: no wage becomes 0
replace inc_wage = . if inc_wage == -1        // then: unknown becomes missing
```

Be careful

In the opposite order, the -1 become missing in the first line and **zero** in the second: the unknown wages disappear without a trace.

Good practice

After every `replace`, read Stata's message: `(3 real changes made)`. Know in advance how many changes you expect. `(0 real changes made)` almost always means the condition is wrong.

Conditions on two variables

For total wealth, a non-owner has 0 in housing; an owner who does not know the value is missing.

```
replace home_value = 0 if home_value == -2 & owns_home == 0   // 4 real changes
replace home_value = . if home_value == -1 & owns_home == 1   // 1 real change
```

Common error

Writing the wrong variable in the condition (`if inc_wage == -2` instead of `if home_value == -2`) gives `(0 real changes made)` and no error. Check that the variable in the condition is the one you mean to change.

Good practice

Read the survey documentation: for each question it states to whom it is asked (its *filter*). That is what tells you whether a missing value is "not applicable" or "does not know".

Loops: doing the same thing for several variables

```
foreach v of varlist inc_wage inc_pension {
    count if missing(`v')                    // how many before
    replace `v' = 0 if missing(`v')          // no income becomes 0
    replace `v' = . if `v' == -1             // unknown becomes missing
}
```

- Round 1: Stata replaces ‘v’ by `inc_wage` and runs the block. Round 2: by `inc_pension`.
- It is the same as writing the block twice, changing only the name.
- On a Spanish Mac keyboard: ‘ is the key to the right of **P**; ’ the key to the right of **0**.

Common error

The { must close the first line and the } must stand on its own line. Code on the same line as a brace stops the loop.

Local macros live only during one run

```
local components inc_wage inc_pension   // store the text
summarize `components'                  // Stata reads: summarize inc_wage inc_pension
```

Common error

If you run the `local` line and then the `summarize` line **separately**, the macro is empty: Stata runs `summarize` alone and summarises *every* variable, with no error. Run the definition and its use in the same selection.

Good practice

Indent the commands inside a loop by four spaces. Stata does not need it, but you will.

Adding components: + or `rowtotal`?

```
generate inc_person1 = inc_wage + inc_pension                    // missing if ANY is missing
egen     inc_person2 = rowtotal(inc_wage inc_pension), missing   // missing only if ALL are
```

Person 2 of household 1: wage unknown, pension 0. `inc_person1` = `.` `inc_person2` = 0.

Key idea

- `rowtotal` treats missing as zero. Right when missing means "does not have it".
- + keeps the total missing. Right when missing means "does not know".

That is why "does not have it" is converted to 0 *before* adding.

From persons to households

```
bysort hh_id: egen inc_hh    = total(inc_person1)            // sum over members
bysort hh_id: egen n_unknown = total(missing(inc_person1))   // members with unknown income
replace inc_hh = . if n_unknown > 0                          // unknown member -> unknown
    household
bysort hh_id: egen n_members = count(person_id)               // household size
```

- `bysort hh_id:` runs the command separately for each household and writes the result on every row of that household.
- `total(missing(x))` adds the 1s of `missing(x)`: the number of members with unknown `x`.

Be careful

`total()`, like `rowtotal`, treats missing as zero. Without the second and third lines, household 1 gets 28,000: only the member we know.

One row per household, and check it

```
bysort hh_id: keep if _n == 1   // _n = row number within each household
isid hh_id                      // silent if hh_id identifies each row
count                           // 6 households
```

- Compute household variables such as size **before** `keep`: afterwards the members are gone.
- `isid` says **nothing** when everything is right, and stops with an error if a household appears twice. Silence is the good news.

Good practice

A do-file that stops with an error is better than one that silently produces wrong numbers. Put a check after every step that changes the structure of the data.

Your checking toolkit

Tool	What it tells you
`(n real changes made)`	How many values a `replace` changed.
`list x y, noobs abbreviate(12)`	The values, in one readable table.
`count`, `count if ...`	How many rows (meet a condition).
`isid` *vars*	Whether *vars* identify each row (silent if yes).
`assert` *condition*	Stops if the condition is false in any row.
`codebook x`	Type of `x` and number of missing values.
`return list`	The results stored by the last command.

Good practice

Do not overwrite original survey variables: create new ones and label them, `label variable inc_hh "Household income, EUR"`.

Weighted and unweighted means

```
summarize inc_hh                    // Obs 4, mean 27,250
summarize inc_hh [aw = weight]      // Obs 4, weight 5,500, mean 25,618
```

- **Obs = 4, not 6**: households with missing income are dropped without warning. The mean is only for households whose income we know; this is why the EFF and the HFCS impute missing values.
- The weighted mean is lower: the poorest household has the largest weight.
- `Weight = 5,500`: the number of population households these 4 represent.

Key idea

Always look at `Obs` before looking at `Mean`.

Confidential microdata

Rules of the game

1. **Read, never write, the original files.**
2. **Only aggregated results leave** the protected environment: means, medians, shares, coefficients. Never individual records.
3. **Keep intermediate microdata in memory**, with `preserve` and `restore`, not in new files.
4. **Count the observations behind each number.** Providers often set a minimum per published figure.

Be careful

A log records everything printed on screen. A `list` with a log open copies individual records into it.

Paths: write them once

```
global raw "/secure/eff/2022"          // original data: read only
global out "/Users/me/project/output" // your results
use "$raw/file.dta", clear             // $raw is replaced by the path
```

- A *global* macro stores text, like a local, but survives between runs until Stata is closed.
- Write it with `$name`, without backtick and apostrophe.

Good practice

Define all paths at the top of a master do-file. If the data move, you change one line instead of every `use` in the project.

A mock survey with five implicates

```
clear
input hh_id str1 region age_group weight income owns_house house_value
1 "N" 1 1200 18000 0      .
2 "N" 2  900 42000 1 210000
3 "N" 3 1500     . 1 150000
4 "S" 1 2000 15000 0      .
5 "S" 2 1100 39000 1 180000
6 "S" 3  700     . 1 320000
7 "S" 2 1300 27000 1      .
8 "N" 3  600 55000 1 400000
end
```

Eight households in two regions. Three values were not reported: the income of households 3 and 6, and the home value of household 7. Households 1 and 4 do not own a home: their missing home value means "not applicable".

The provider imputes five times

```
forvalues m = 1/5 {                                    // the provider would do this
    preserve
    replace income      = 30000 + 1000*`m'  if hh_id == 3
    replace income      = 22000 + 1500*`m'  if hh_id == 6
    replace house_value = 200000 + 5000*`m' if hh_id == 7
    save toy_imp`m', replace                           // toy_imp1.dta ... toy_imp5.dta
    restore
}
```

Five files, identical except for the three imputed values. Run it once, in a practice folder.

Stacking the implicates: `append`

```
use toy_imp1, clear                                  // implicate 1 in memory
generate implicate = 1                               // mark its rows
forvalues m = 2/5 {
    append using toy_imp`m'                          // add implicate m below
    replace implicate = `m' if missing(implicate)    // mark the new rows
}
isid hh_id implicate                                 // household x implicate is unique
tabulate implicate                                   // 8 rows in each
```

Key idea

Each household now appears five times. It is identified by `hh_id` **and** `implicate` together (and, in the HFCS, the country as well).

Labels and dummies

```
label define age_lbl 1 "Under 40" 2 "40-64" 3 "65 and over"   // the list of texts
label values age_group age_lbl                                 // attach it to the variable
tabulate age_group, generate(age_)                             // dummies age_1, age_2, age_3
```

- Labels change how codes are displayed, not the codes. Conditions still use the codes: `if age_group == 3`.
- The mean of a 0/1 dummy is a share: `age_3` gives the share of households aged 65 and over.

Common error

Text values need double quotes: `if region == "N"`. Without them Stata looks for a variable called `N` (`r(111)`); comparing text with a number gives `type mismatch r(109)`.

One statistic per implicate, then the average

$$\bar{\theta} = \frac{1}{5} \sum_{m=1}^{5} \hat{\theta}_m$$

```
forvalues m = 1/5 {
    quietly summarize income [aw = weight] if implicate == `m', detail   // compute, do not
     print
    scalar mean_`m' = r(mean)                                            // keep the mean
    scalar p50_`m'  = r(p50)                                             // keep the median
}
display "Mean:   " (mean_1 + mean_2 + mean_3 + mean_4 + mean_5) / 5
display "Median: " (p50_1 + p50_2 + p50_3 + p50_4 + p50_5) / 5
```

- `summarize, detail` stores results in `r()`: `r(mean)`, `r(p50)` (`return list` shows all).
- `quietly`: runs the command, stores results, prints nothing.
- `scalar`: keeps a number, because `r()` is overwritten by the next command.

aw or pw?

- Survey weights are **probability weights** (pw): the inverse of the probability of being sampled.
- For a point estimate, **analytic weights** (aw) give exactly the same number. They differ only in standard errors.
- Rule:
  - only the number (summarize, table, collapse) ⇒ aw;
  - standard errors (svy, mi estimate, regressions) ⇒ pw.

Common error

pweights not allowed r(101): summarize does not accept pw.

Be careful

Never compute a statistic on the five implicates pooled and call it the result: for a median it is generally different, and standard errors are wrong in all cases.

A quick table by implicate

```
table implicate [aw = weight], statistic(mean income) statistic(median income) nototals
```

- `table implicate`: one row per implicate.
- `[aw = weight]`: weighted.
- each `statistic()`: one column.
- `nototals`: no "Total" row, which would pool the implicates.

Requires Stata 17 or later. It only displays: nothing is stored.

Results as data: `preserve`, `collapse`, `restore`

```
preserve                                                    // snapshot of the 40 rows
collapse (mean) mean_inc = income (median) p50_inc = income ///
    [aw = weight], by(implicate)                            // 5 rows, one per implicate
collapse (mean) mean_inc p50_inc                            // 1 row: average of the five
list
restore                                                     // back to the 40 rows
```

- `(mean) mean_inc = income`: statistic, new name, original variable.
- By group: `by(implicate age_group)` first, then `by(age_group)`.

Common error

`preserve` lasts only while the selection runs: at the end Stata restores the data automatically. Run each block from `preserve` to `restore` **in one selection**.

Standard errors with `mi`: implicate 0

`mi` expects the original data as implicate 0. It is not used in the calculations, so a copy of implicate 1 will do.

```
preserve
expand 2 if implicate == 1, generate(copy)   // duplicate implicate 1; copy = 1 on new rows
replace implicate = 0 if copy == 1           // copies become implicate 0
drop copy
tabulate implicate                           // 8 rows in each of 0, 1, ..., 5
```

- `expand 2` turns each selected row into two and puts the copies at the end of the data.
- `generate(copy)` marks originals with 0 and copies with 1.
- The Banco de España code book does the same with a helper variable: `gen aux = 1`, `replace aux = 2 if imp == 1`, `expand aux`.

Standard errors with `mi`: declare and estimate

```
mi import flong, id(hh_id) m(implicate) clear      // household id, implicate id
mi svyset [pweight = weight]                       // pw: we want standard errors
mi estimate: svy: mean income                      // mean and its standard error
mi estimate: svy: mean income, over(age_group)     // by group
mi estimate: svy: proportion owns_house            // shares
restore
```

`Coefficient` is the point estimate (equal to the `collapse` result); `Std. err.` is its standard error.

Common error

Run everything from `preserve` (previous slide) to `restore` in one selection; otherwise implicate 0 disappears and `mi import` fails.

Key idea

In the HFCS, add the replicate weights: `mi svyset [pw = w], bsrweight(wr0001-wr1000) vce(bootstrap)`.

Taking results out

```
preserve
collapse (mean) mean_inc = income [aw = weight], by(implicate age_group)
collapse (mean) mean_inc, by(age_group)
export excel using "results_income.xlsx", firstrow(variables) replace
restore
```

`firstrow(variables)`: variable names in the first row; `replace`: overwrite an earlier version.

Be careful

Before exporting, count the households behind each figure: `tabulate age_group if implicate == 1`.

Good practice

One task per do-file (`01_build.do`, `02_stats.do`, `03_export.do`) and a master do-file that runs them in order with `do`. One file reproduces every result.

Summary: twelve commands for survey microdata

Command	Use
`append`	Stack the implicate files.
`label define/values`	Readable texts for numeric codes.
`tabulate, generate()`	One dummy per category (shares).
`summarize, detail`	Weighted mean and percentiles in `r()`.
`quietly`	Run without printing.
`scalar`	Keep one number.
`table`	Statistics by implicate or group, on screen.
`preserve/restore`	Protect the microdata in memory.
`collapse`	Microdata into a dataset of statistics.
`expand`	Duplicate rows (implicate 0).
`mi import/svyset/estimate`	Estimates and standard errors with five implicates.
`export excel`	Take aggregated results out.

References

Antón, J.-I. *Introducción al análisis de datos con el software Stata*. Course materials, Universidad de Salamanca. `https://jianton.github.io/curso_stata.html`

Banco de España. *Encuesta Financiera de las Familias: Libro de Código*. Madrid: Banco de España.

Household Finance and Consumption Network (2026). *The Household Finance and Consumption Survey: Methodological report for the 2023 wave*. ECB Statistics Paper Series No 52.

Long, J. S. (2009). *The Workflow of Data Analysis Using Stata*. College Station, TX: Stata Press.

StataCorp. *Economics: Features for economists*.
`https://www.stata.com/disciplines/economics/`

StataCorp. *Stata Reference Manuals*: `append`, `collapse`, `egen`, `forvalues`, `mi estimate`.
`https://www.stata.com/manuals/`

UCLA Office of Advanced Research Computing. *Stata resources and learning modules*.
`https://stats.oarc.ucla.edu/stata/`